

TECNICATURA UNIVERSITARIA en PROGRAMACION.

Cátedra: Base de Datos II.

Año: 2026

Integrantes:

- Hotchkyss, Gersom Tomás
 - Legajo: 30360
 - Correo: gersomtomasosky@gmail.com
- Burgos Salvadeo, Luka Edgardo
 - Legajo: 30288
 - Correo: lukyta2013@gmail.com
- Romero, Germán
 - Legajo: 30277
 - Correo: yermanpc153@gmail.com

Índice

Etapla 1: Fundación Relacional.....	3
Introducción	3
Descripción general del sistema	3
Modelo Entidad-Relación	3
Actores del sistema	4
Requisitos funcionales	4
Requisitos no funcionales	4
Identificación de entidades y relaciones	5
Reglas de negocio	6
Justificación de decisiones técnicas iniciales.....	6
Modelo Relacional	6
Implementación en phpMyAdmin	7
Etapla 2: Semiestructurados	8
Motivación para el uso de JSON.....	8
Evolución del modelo: sentencias ALTER TABLE	8
Estructura de los campos JSON	9
Carga de datos JSON en los registros existentes.....	10
Consultas SQL con funciones JSON.....	10
Ventajas y limitaciones del enfoque	12
Etapla 3: Migración a NoSQL (Documental)	13
Selección del Subconjunto del Modelo Relacional.....	13
Análisis de las entidades principales	13
Aplicación del árbol de decisión (embedding vs referencing).....	13
Campos multivaluados o tipo tags	14
Justificación del modelo documental	14
Modelo documental resultante.....	14
Campo precomputado	15
Implementación en MongoDB Atlas.....	15
Comparativa SQL VS MongoDB.....	17
Conclusión.....	18

Etapas 1: Fundación Relacional

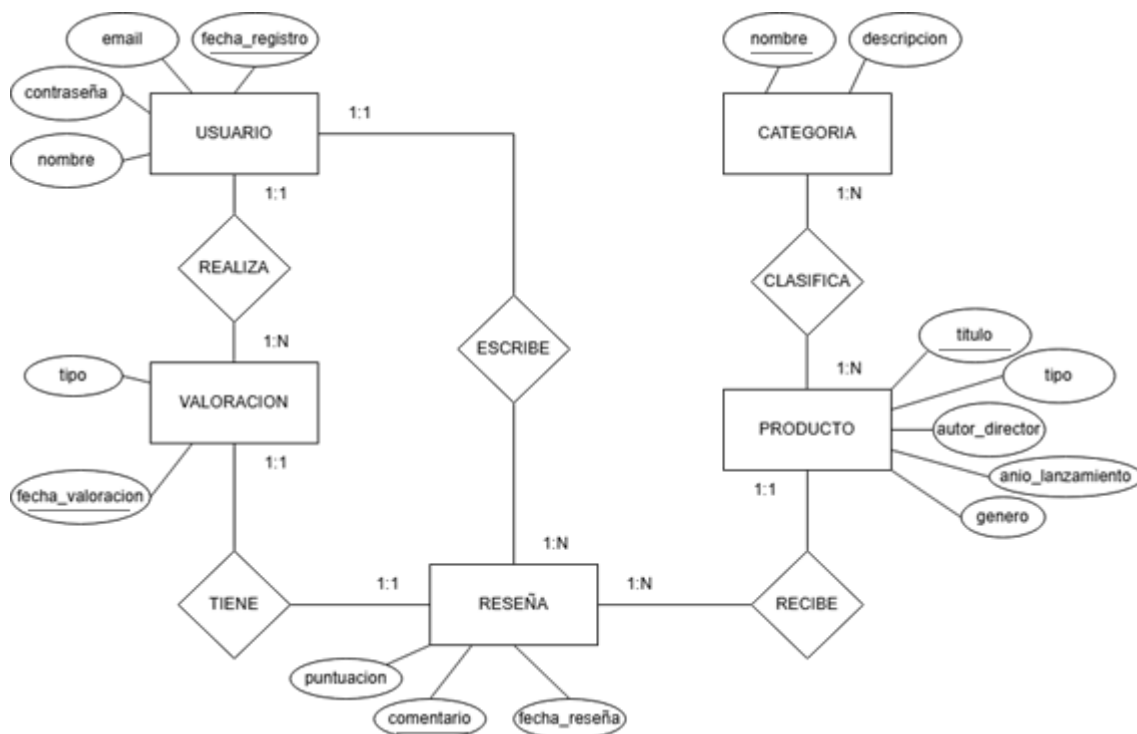
Introducción:

El caso de estudio consiste en el desarrollo de una plataforma digital de reseñas de productos culturales, como películas y libros, inspirada en sitios existentes. A partir de estos ejemplos, se definen los datos involucrados, los usuarios que interactúan y las funciones esperadas del sistema.

Descripción del sistema:

La plataforma permite a usuarios registrados explorar un catálogo, escribir reseñas con puntuación y comentario, y valorar reseñas de otros usuarios con “me gusta” o “no me gusta”. Además, los productos se organizan en categorías para facilitar la navegación y el filtrado. El sistema busca centralizar opiniones, destacar las reseñas más relevantes y ofrecer información estructurada para consultas sobre valoraciones, interacción y preferencias de los usuarios.

Modelo Entidad-Relación



Actores del sistema

- **Usuario registrado:** Es el actor principal. Puede explorar el catálogo, escribir, editar y eliminar reseñas, así como valorar las de otros usuarios. El sistema guarda su historial de actividad y preferencias, lo que permite futuras recomendaciones.
- **Administrador del sistema:** Se encarga de gestionar el catálogo y las categorías, agregar o corregir productos y moderar reseñas. Aunque no se modela como entidad separada, sus funciones están contempladas en el diseño.
- **Visitante anónimo:** Puede consultar el catálogo y leer reseñas sin registrarse, pero no puede interactuar activamente. No genera datos en el sistema ni requiere modelado específico.

Requisitos funcionales

Módulo	Restricción clave
Usuarios	Email único; fecha de registro; contraseña hasheada.
Productos	Tipo ENUM ('película'/ 'libro'); año entre 1888 y 2100; atributos variables en JSON.
Categorías	Nombre único; un producto puede pertenecer a varias categorías (N:M).
Reseñas	Máximo una por usuario por producto (UNIQUE); puntuación 1–10 (CHECK).
Valoraciones	Una por usuario por reseña (UNIQUE); no se puede valorar la propia reseña.

Requisitos no funcionales: Los requisitos no funcionales definen las características de calidad que debe cumplir el sistema, más allá de su comportamiento funcional.

Atributo	Descripción
Integridad referencial	Toda reseña debe estar asociada a un usuario y un producto existentes. Al eliminar un usuario o producto, sus reseñas se eliminan en cascada para mantener la consistencia.

Unicidad	No pueden existir usuarios con el mismo email, categorías con el mismo nombre ni reseñas duplicadas de un mismo usuario para un producto.
Validación de datos	La puntuación debe estar entre 1 y 10, y el año de lanzamiento dentro de un rango válido (1888–2100).
Escalabilidad	El sistema debe soportar el crecimiento de usuarios y productos sin afectar el rendimiento ni la integridad.
Trazabilidad temporal	Se deben registrar con precisión las fechas de creación de usuarios, reseñas y valoraciones.

Identificación de entidades y relaciones: A partir del análisis anterior, identificamos las siguientes entidades principales del dominio y las relaciones que existen entre ellas. Esta identificación es el puente entre el análisis de requisitos y el diseño del modelo entidad-relación.

Entidades principales

Entidad	Atributos clave	Justificación
USUARIO	id, nombre, email, contraseña, fecha_registro	Representa a toda persona que interactúa activamente con la plataforma.
PRODUCTO	id, título, tipo, autor/director, año, género	Centraliza películas y libros. Se utiliza tipo como discriminador y se vincula con la categoría mediante una clave foránea.
CATEGORÍA	id, nombre, descripción	Permite clasificar los productos en múltiples grupos temáticos.
RESEÑA	id, puntuación, comentario, fecha	Entidad central del sistema. Conecta un usuario con un producto y captura su opinión estructurada.
VALORACIÓN	id, tipo (like/dislike), fecha_hora	Registra la interacción de un usuario con la reseña de otro, permitiendo medir la utilidad percibida de cada reseña.

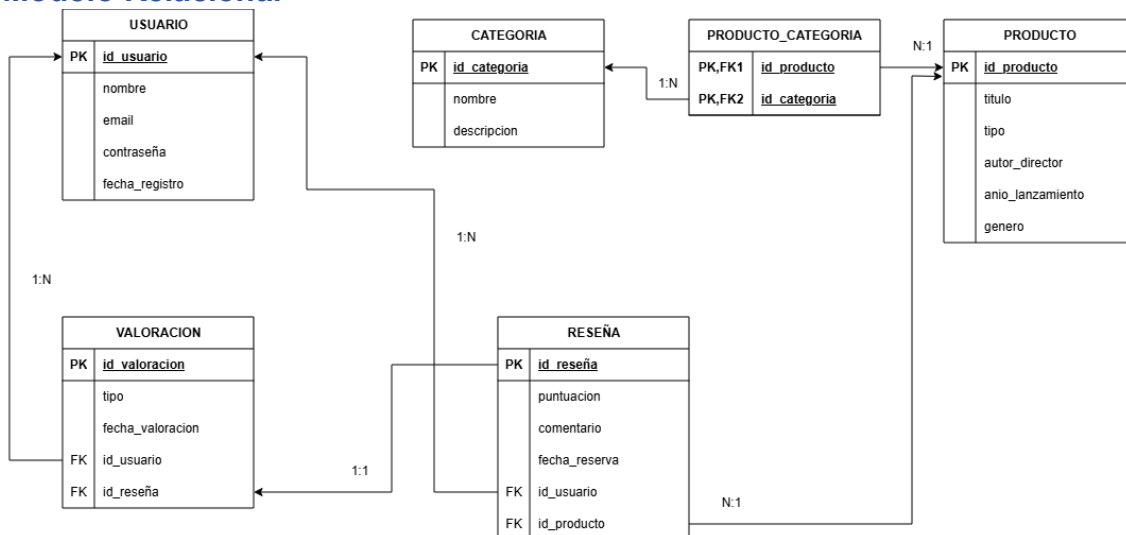
Reglas de negocio

Mecanismo	Regla
UNIQUE	Email irrepetible; nombre de categoría único; 1 reseña por usuario/producto; 1 valoración por usuario/reseña.
CHECK	Puntuación entre 1 y 10; año de lanzamiento entre 1888 y 2100.
ENUM	Tipo de producto: 'pelicula' o 'libro'. Tipo de valoración: 'like' o 'dislike'.
FOREIGN KEY / ON DELETE CASCADE	Al eliminar un usuario se borran sus reseñas y valoraciones; al eliminar un producto se borran sus reseñas; al eliminar una reseña se borran sus valoraciones.

Justificación de decisiones de diseño

- **Tabla única PRODUCTO:** películas y libros se unifican porque comparten la mayoría de atributos y relaciones (reseñas, categorías). Esto evita duplicar lógica. Las diferencias se manejan con un campo JSON.
- **Uso de JSON para atributos variables:** permite almacenar propiedades específicas de cada tipo (ej. duración o ISBN) sin generar muchas columnas nulas ni modificar el esquema.
- **Restricción UNIQUE en RESEÑA (id_usuario, id_producto):** garantiza que cada usuario solo pueda reseñar una vez cada producto, evitando manipulaciones y asegurando integridad a nivel de base de datos.

Modelo Relacional



Implementación SQL en phpMyAdmin

Script completo — base de datos, tablas, índices y datos de ejemplo:

Crear base de datos

```
CREATE DATABASE IF NOT EXISTS resenas_db  
  
CHARACTER SET utf8mb4  
  
COLLATE utf8mb4_unicode_ci;  
  
USE resenas_db;
```

Tabla USUARIO

```
CREATE TABLE usuario (  
    id_usuario    INT          PRIMARY KEY AUTO_INCREMENT,  
    nombre        VARCHAR(100) NOT NULL,  
    email         VARCHAR(150) NOT NULL UNIQUE,  
    contraseña    VARCHAR(255) NOT NULL,  
    fecha_registro DATE        NOT NULL DEFAULT (CURDATE())  
) ENGINE=InnoDB;
```

Tabla CATEGORÍA

```
CREATE TABLE categoria (  
    id_categoria INT          PRIMARY KEY AUTO_INCREMENT,  
    nombre       VARCHAR(80)  NOT NULL UNIQUE,  
    descripcion  VARCHAR(255)  
) ENGINE=InnoDB;
```

INSERT — USUARIO

```
INSERT INTO usuario (nombre, email, contraseña, fecha_registro) VALUES  
(  
    'Ana García', 'ana@mail.com', 'hash1', '2023-01-10'),  
    ('Carlos López', 'carlos@mail.com', 'hash2', '2023-03-22'),  
    ('María Fernández', 'maria@mail.com', 'hash3', '2023-05-14'),  
    ('Juan Pérez', 'juan@mail.com', 'hash4', '2023-07-30'),  
    ('Laura Rodríguez', 'laura@mail.com', 'hash5', '2023-09-05'),  
    ('Diego Martínez', 'diego@mail.com', 'hash6', '2024-01-18');
```

Etapas 2: Semiestructurados

Motivación para el uso de JSON

El sistema maneja dos tipos de productos (películas y libros) con atributos distintos entre sí. Modelarlos en columnas fijas genera valores NULL masivos sin sentido semántico:

Atributo	Inception	1984	Harry Potter
duracion_min	148	NULL	NULL
clasificacion	PG-13	NULL	NULL
isbn	NULL	978-0-452...	978-0-747...
paginas	NULL	328	223
editorial	NULL	Secker & Warburg	Bloomsbury
saga	NULL	NULL	Harry Potter

Una tabla separada por tipo duplicaría toda la lógica de relaciones (reseñas, categorías) sin beneficio conceptual. El campo JSON es la solución: mantiene la unidad de la entidad PRODUCTO y permite extender atributos sin modificar el esquema.

Lo mismo aplica a las reseñas, cuyos metadatos (idioma, spoilers, fuente, tiempo de lectura) son opcionales y pueden crecer con el tiempo. Cada nueva necesidad sin JSON implicaría un ALTER TABLE en producción.

Evolución del modelo: ALTER TABLE

Agregar preferencias a usuario

ALTER TABLE usuario

ADD COLUMN preferencias JSON NULL;

Agregar atributos_extra a producto

ALTER TABLE producto

ADD COLUMN atributos_extra JSON NULL;

Agregar metadatos a resena

ALTER TABLE resena

ADD COLUMN metadatos JSON NULL;

Se declaran NULL porque al agregar columnas sobre filas existentes MySQL no puede asignar un valor JSON automático. Los valores se cargan luego con UPDATE.

Estructura de los campos JSON: A continuación se documenta la estructura de datos que almacena cada campo JSON, con ejemplos concretos de los valores insertados en los registros de prueba.

producto atributos_extra

Para películas:

```
{
  "duracion_min": 148,
  "idioma_original": "en",
  "clasificacion": "PG-13",
  "premios": ["Oscar"]
}
```

Para libros:

```
{
  "isbn": "978-0-452-28423-4",
  "paginas": 328,
  "editorial": "Secker & Warburg",
  "edicion": "primera"
}
```

resena metadatos

```
{
  "idioma": "es",
  "contiene_spoiler": false,
  "fuente": "web",
  "tiempo_lectura_min": 3
}
```

usuario preferencias

```
{
```

```
"generos_favoritos": ["Drama", "Ciencia Ficción"],
"idioma": "es",
"notificaciones": true
}
```

Carga de datos

Cargar atributos_extra en productos

```
UPDATE producto SET atributos_extra =
'{"duracion_min":148,"idioma_original":"en","clasificacion":"PG-
13","premios":["Oscar"]}'
WHERE id_producto = 1;
```

```
UPDATE producto SET atributos_extra = '{"isbn":"978-0-452-28423-
4","paginas":328,"editorial":"Secker & Warburg","edicion":"primera"}'
WHERE id_producto = 3;
```

Cargar metadatos en reseñas

```
UPDATE resena SET metadatos =
'{"idioma":"es","contiene_spoiler":false,"fuente":"web","tiempo_lectura_min":3}'
WHERE id_resena = 1;
```

Cargar preferencias en usuarios

```
UPDATE usuario SET preferencias = '{"generos_favoritos":["Drama","Ciencia
Ficción"],"idioma":"es","notificaciones":true}'
WHERE id_usuario = 1;
```

Consultas SQL con funciones JSON: Se implementaron seis consultas que demuestran el uso de las principales funciones JSON de MySQL 8. Cada una responde a una pregunta de negocio concreta y aplica una función distinta.

N°	Función principal	Propósito	Pregunta de negocio
1	JSON_EXTRACT + CAST	Filtrar por valor numérico en JSON	¿Qué películas duran más de 130 minutos?

2	JSON_EXTRACT + JSON_UNQUOTE	Filtrar por múltiples claves JSON	¿Qué reseñas sin spoilers vienen de la web?
3	JSON_CONTAINS	Buscar valor dentro de array JSON	¿Qué usuarios prefieren el género Fantasía?
4	JSON_SET	Modificar claves sin reemplazar el JSON	¿Cómo marcar una reseña como verificada?
5	JSON_VALUE	Extracción directa con sintaxis simplificada	¿Qué libros tienen ISBN registrado?
6	AVG + COUNT + CASE	Ranking combinando datos relacionales	¿Cuál es el ranking de productos por puntuación?

Algunos ejemplos de las consultas ejecutadas:

Películas con duración mayor a 130 minutos:

```
SELECT
    titulo,
    tipo,
    JSON_UNQUOTE(JSON_EXTRACT(atributos_extra, '$.duracion_min')) AS
    duracion_min,
    JSON_UNQUOTE(JSON_EXTRACT(atributos_extra, '$.clasificacion')) AS
    clasificacion
FROM producto
WHERE tipo = 'pelicula'
    AND CAST(JSON_EXTRACT(atributos_extra, '$.duracion_min') AS UNSIGNED) >
130;
```

Usuarios que prefieren el género Fantasía:

```
SELECT
    nombre,
    email,
    JSON_UNQUOTE(JSON_EXTRACT(preferencias, '$.idioma')) AS
    idioma,
    JSON_EXTRACT(preferencias, '$.generos_favoritos') AS
    generos_favoritos
FROM usuario
WHERE JSON_CONTAINS(preferencias, '"Fantasía"', '$.generos_favoritos');
```

Marcar una reseña como verificada:

```
UPDATE resena
```

```
SET metadatos = JSON_SET(  
  metadatos,  
  '$.verificada', true,  
  '$.fecha_verificacion', '2024-04-01'  
)
```

Ventajas y limitaciones del enfoque:

- **Ventajas:** el esquema no necesita modificarse ante nuevos atributos opcionales; se eliminan columnas nulas sin sentido semántico; MySQL 8 soporta índices funcionales sobre claves JSON.
- **Limitaciones:** no se pueden definir restricciones CHECK ni FOREIGN KEY dentro de un JSON; las consultas son más verbosas que sobre columnas regulares; la validación de estructura queda a cargo de la aplicación.

Etapa 3: Migración a NoSQL (Documental)

PARTE A:

Selección del Subconjunto del Modelo Relacional

Para la migración al paradigma NoSQL orientado a documentos, se seleccionó el módulo principal del sistema de interacción compuesto por las entidades Productos, Categorías, Reseñas, Usuarios y Valoraciones. En el modelo relacional original, obtener la información completa de un producto con sus reseñas, autores de las reseñas y valoraciones de otros usuarios requería múltiples operaciones JOIN entre distintas tablas:

- producto
- categoria (mediante la tabla intermedia producto_categoria)
- resena
- usuario
- valoracion

Este subconjunto fue elegido porque representa el núcleo funcional del sistema y posee un patrón de acceso fuertemente orientado a lectura (read-heavy), ideal para un modelo documental desnormalizado.

ANÁLISIS DE LAS ENTIDADES PRINCIPALES

1. Producto

La entidad Producto representa películas y libros disponibles dentro de la plataforma.

- Los datos de un producto normalmente se leen junto con categorías, reseñas y valoraciones.
- Un producto puede tener muchas reseñas (1:N) y varias categorías (N:M).
- Las categorías son compartidas entre distintos productos, mientras que las reseñas son exclusivas del producto.
- La información del producto se actualiza poco, pero se consulta frecuentemente.

2. Reseña

La entidad Reseña almacena las opiniones realizadas por los usuarios sobre productos.

- Las reseñas siempre se leen dentro del contexto de un producto.
- Una reseña puede tener muchas valoraciones (likes/dislikes).
- La reseña pertenece exclusivamente a un producto y un usuario.
- Las reseñas tienen alta frecuencia de lectura y actualizaciones moderadas debido a las valoraciones.

3. Usuario

La entidad Usuario representa las personas registradas en la plataforma.

- Generalmente se leen datos básicos del usuario junto con las reseñas.
- Un usuario puede crear muchas reseñas y muchas valoraciones.
- Los usuarios son entidades compartidas por distintos productos y reseñas.
- La información del usuario se actualiza poco y se consulta frecuentemente.

APLICACIÓN DEL ÁRBOL DE DECISIÓN (EMBEDDING VS REFERENCING)

Relación: Producto → Reseñas

Decisión: Embedding

Justificación: Las reseñas se consultan casi siempre junto al producto y pertenecen exclusivamente a él.

Relación: Reseña → Valoraciones

Decisión: Embedding

Justificación: Las valoraciones son pequeñas, dependen totalmente de la reseña y se leen junto con ella.

Relación: Producto → Categorías

Decisión: Embedding parcial

Justificación: Las categorías son pocas y útiles para filtrar productos rápidamente.

Relación: Reseña → Usuario

Decisión: Embedding parcial

Justificación: Solo se almacenan datos básicos del usuario dentro de la reseña para evitar JOINS innecesarios.

Relación: Usuario completo

Decisión: Referencing

Justificación: El usuario es una entidad compartida entre muchas reseñas y valoraciones.

CAMPOS MULTIVALUADOS O TIPO TAGS

El sistema posee campos similares a etiquetas o tags, principalmente:

- categorías

- genero

En MongoDB se modelan mediante arrays de subdocumentos embebidos:

```
"categorias": [  
  {  
    "id_categoria": 1,  
    "nombre": "Ciencia Ficción"  
  },  
  {  
    "id_categoria": 4,  
    "nombre": "Fantasía"  
  }  
]
```

Este enfoque permite flexibilidad, consultas eficientes y evita tablas intermedias.

JUSTIFICACIÓN DEL MODELO DOCUMENTAL

La migración a MongoDB se fundamenta en el análisis de los patrones de acceso a la información. El módulo de productos y reseñas posee una carga orientada principalmente a lectura, donde la prioridad es recuperar rápidamente toda la información relacionada con un producto.

El modelo documental permite desnormalizar los datos y agrupar en un único documento toda la información necesaria para representar un producto completo, incluyendo categorías, reseñas y valoraciones. Esto elimina la necesidad de múltiples operaciones JOIN y simplifica significativamente las consultas.

Además, MongoDB ofrece flexibilidad de esquema, permitiendo almacenar distintos atributos dependiendo del tipo de producto. Por ejemplo, una película puede tener duración y clasificación, mientras que un libro puede contener ISBN y cantidad de páginas.

MODELO DOCUMENTAL RESULTANTE

La colección principal del sistema es productos.

Ejemplo de documento:

```
{  
  "_id": 1,  
  "titulo": "Inception",  
  "tipo": "pelicula",  
  "autor_director": "Christopher Nolan",  
  "anio_lanzamiento": 2010,  
  "genero": "Ciencia Ficción",  
  
  "categorias": [  
    {  
      "id_categoria": 1,  
      "nombre": "Ciencia Ficción"  
    }  
  ],  
}
```

```
"atributos_extra": {
  "premios": ["Oscar"],
  "duracion_min": 148,
  "clasificacion": "PG-13",
  "idioma_original": "en"
},

"resenas": [
  {
    "id_resena": 1,
    "puntuacion": 9,
    "comentario": "Una obra maestra del cine moderno.",
    "fecha_resena": "2024-01-15",

    "usuario": {
      "id_usuario": 1,
      "nombre": "Ana García",
      "email": "ana@mail.com"
    },

    "valoraciones": [
      {
        "usuario": "Carlos López",
        "tipo": "like"
      },
      {
        "usuario": "María Fernández",
        "tipo": "like"
      }
    ]
  }
],

"promedio_puntuacion": 9.0
}
```

CAMPO PRECOMPUTADO

El campo promedio_puntuacion se almacena directamente dentro del documento del producto.

Justificación:

Este valor evita recalcular constantemente el promedio de puntuaciones mediante aggregation pipelines, mejorando el rendimiento y acelerando las consultas frecuentes.

PARTE B:

IMPLEMENTACIÓN EN MONGODB ATLAS

Paso 1: Configuración de Atlas

Se creó un clúster gratuito M0 en MongoDB Atlas utilizando AWS en la región de São Paulo (sa-east-1) para minimizar la latencia respecto al entorno local de desarrollo.

Paso 2: Conexión con MongoDB Compass

Se utilizó la cadena de conexión provista por Atlas para establecer conexión mediante MongoDB Compass y administrar la base de datos resenas_db_nosql.

Paso 3: Inserción de documentos

Se insertaron 7 documentos en la colección productos utilizando insertMany().

Además, se aprovecharon las capacidades schema-less de MongoDB permitiendo que

algunos documentos posean atributos diferentes dependiendo del tipo de producto.

Ejemplo:

- Las películas contienen duracion_min y clasificacion.
- Los libros contienen isbn y paginas.

OPERACIONES EN MONGODB

a) findOne() utilizando dot notation

```
db.productos.findOne({
  "resenas.usuario.nombre": "Ana García"
})
```

b) find() utilizando \$in sobre arrays

```
db.productos.find({
  "categorias.nombre": {
    $in: ["Terror", "Thriller"]
  }
})
```

c) updateOne() utilizando \$push

```
db.productos.updateOne(
  { _id: 1 },
  {
    $push: {
      "resenas.0.valoraciones": {
        usuario: "Laura Rodríguez",
        tipo: "like"
      }
    }
  }
)
```

d) updateOne() utilizando operador positional \$

```
db.productos.updateOne(
  { "resenas.id_resena": 1 },
  {
    $set: {
      "resenas.$.puntuacion": 10
    }
  }
)
```

e) Aggregation Pipeline

```
db.productos.aggregate([
  {
    $match: {
      tipo: "pelicula"
    }
  },
  {
    $unwind: "$resenas"
  },
  {
    $group: {
      _id: "$genero",
      promedio: {
        $avg: "$resenas.puntuacion"
      }
    }
  }
])
```



```

    }
  }
},
{
  $sort: {
    promedio: -1
  }
}
]
])

```

PARTE C:

COMPARATIVA SQL VS MONGODB

Consulta 1: Buscar productos de una categoría específica

SQL:

```

SELECT p.id_producto, p.titulo, p.tipo
FROM producto p
JOIN producto_categoria pc
ON p.id_producto = pc.id_producto
JOIN categoria c
ON pc.id_categoria = c.id_categoria
WHERE c.nombre = 'Ciencia Ficción';

```

MongoDB:

```

db.productos.find(
  { "categorias.nombre": "Ciencia Ficción" },
  { titulo: 1, tipo: 1 }
)

```

Comparación:

MongoDB resulta más simple y eficiente para este caso porque las categorías ya están embebidas dentro del documento.

Consulta 2: Obtener reseñas con datos del usuario

SQL:

```

SELECT r.id_resena, r.puntuacion, r.comentario, u.nombre
FROM resena r
JOIN usuario u
ON r.id_usuario = u.id_usuario
WHERE r.id_producto = 1;

```

MongoDB:

```

db.productos.find(
  { _id: 1 },
  {
    "resenas.id_resena": 1,
    "resenas.puntuacion": 1,
    "resenas.comentario": 1,
    "resenas.usuario.nombre": 1
  }
)

```

Comparación:

MongoDB simplifica la consulta al evitar JOINS y almacenar la información relacionada dentro del mismo documento.

Consulta 3: Productos con puntuación perfecta

SQL:

```
SELECT DISTINCT p.titulo  
FROM producto p  
JOIN resena r  
ON p.id_producto = r.id_producto  
WHERE r.puntuacion = 10;
```

MongoDB:

```
db.productos.find(  
  { "resenas.puntuacion": 10 },  
  { titulo: 1, _id: 0 }  
)
```

Comparación:

MongoDB permite realizar consultas directas sobre arrays embebidos mediante dot notation, reduciendo complejidad y cantidad de operaciones.

CONCLUSIÓN

El modelo documental mejora significativamente el rendimiento de lectura al almacenar juntos los datos que normalmente se consultan de manera conjunta. Esto reduce la necesidad de realizar JOINS complejos y simplifica las consultas. MongoDB también aporta flexibilidad gracias a su esquema dinámico, permitiendo que distintos productos posean atributos diferentes sin modificar la estructura general de la base de datos. Los arrays y subdocumentos facilitan representar relaciones multivaluadas como categorías y valoraciones. Sin embargo, SQL continúa siendo una mejor opción para sistemas que requieren integridad transaccional estricta, relaciones altamente complejas y consistencia fuerte entre múltiples entidades. En este proyecto, MongoDB resulta especialmente eficiente para el catálogo y las reseñas, mientras que SQL seguiría siendo conveniente para módulos críticos como autenticación, auditorías o sistemas financieros.